05.05.2025

Układy równań liniowych

Metody numeryczne, projekt 2

Zofia Krzeszowiec
Informatyka, semestr IV, Wydział ETI, Politechnika Gdańska

Wstęp teoretyczny

Rozwiązywanie układów równań liniowych jest bardzo ważnym zagadnieniem w wielu dziedzinach nauki. Odpowiednie dobranie algorytmu ma kluczowy wpływ na czas obliczeń oraz dokładność wyniku. Poniżej opisano 3 metody rozwiązywania równań macierzowych o takiej postaci:

$$Ax = b \tag{1}$$

gdzie:
A - macierz,
b - wektor wyrazów wolnych,
x - wektor niewiadomych.

W opisach wykorzystywane będą następujące oznaczenia:
A = L + U + D,
L - macierz trójkątna dolna, U - macierz trójkątna górna, D - macierz diagonalna.

1. Metody bezpośrednie

Metody bezpośredniego wyznaczania rozwiązania równania charakteryzują się wysoką dokładnością otrzymanego wyniku. Za ich pomocą można otrzymać rezultat w skończonej liczbie operacji algebraicznych[2].

1.1. Faktoryzacja LU

Metoda faktoryzacji LU (ang. LU decomposition) to technika rozkładu macierzy kwadratowej na iloczyn dwóch macierzy:
Równanie jest sprowadzane do następującego układu równań macierzowych:

$$L * y = b \tag{2}$$
$$U * x = y \tag{3}$$

Równanie (2) jest rozwiązywane z wykorzystaniem algorytmu podstawienia w przód (ang. forward substitution), natomiast (3) podstawienia wstecz (ang. back substitution)[1][2][3].

2. Metody iteracyjne

Metody iteracyjne polegają na stopniowym przybliżaniu rozwiązania układu równań liniowych poprzez kolejne iteracje. Zazwyczaj są wykorzystywane w przypadku dużych i rzadkich macierzy, gdzie zastosowanie metod bezpośrednich byłoby zbyt kosztowne obliczeniowo. Cechują się mniejszym zużyciem pamięci, ale ich dokładność oraz szybkość zbieżności zależą od właściwości macierzy układu. Wykonywanie pętli algorytmu kończy się w momencie osiągnięcie pewnej, zadowalającej wartości dokładności wyniku określanej poprzez wyznaczenie normy residuum[2]:

$$r^{(k)} = Ax^{(k)} - b \tag{4}$$

2.1. Metoda Jacobiego

Metoda Jacobiego opiera się na przekształceniu układu równań tak, aby w każdej iteracji aktualizować wartość każdej niewiadomej niezależnie, na podstawie wartości z poprzedniej iteracji. Dla układu równań Ax=b, metoda wykonuje w każdej (k-tej) iteracji działanie (5)[2]:

$$x^{(k+1)} =- D^{-1}(L + U)x^{(k)} + D^{-1}b \tag{5}$$

2.2. Metoda Gaussa–Seidla

Metoda Gaussa–Seidla jest rozwinięciem metody Jacobiego. W przeciwieństwie do niej, w każdej iteracji nowe wartości zmiennych są wykorzystywane natychmiast po ich obliczeniu. Oznacza to, że każda kolejna niewiadoma w bieżącej iteracji bazuje już na zaktualizowanych wartościach poprzednich niewiadomych, co zwykle przyspiesza zbieżność algorytmu.

Iteracyjna postać metody Gaussa–Seidla jest wyrażona wzorem:

$$x^{(k+1)} =- (D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b \tag{6}$$

Opis projektu

W ramach projektu przeprowadzono analizę porównawczą różnych metod rozwiązywania układów równań liniowych w zależności od danych wejściowych. W tym celu zaimplementowano program rozwiązujący równanie macierzowe wykorzystujący metody: Jacobiego, Gaussa-Seidla oraz faktoryzacji LU.

Rozwiązywane równanie definiowane jest następująco:

$$Ax = b, \tag{6}$$

gdzie x jest szukaną wartością,

A jest macierzą systemową o rozmiarze 1207x1207:

$$\begin{bmatrix} a1 & a2 & a3 & 0 & 0 & 0 & 0 & \dots & 0 \\ a2 & a1 & a2 & a3 & 0 & 0 & 0 & \dots & 0 \\ a3 & a2 & a1 & a2 & a3 & 0 & 0 & \dots & 0 \\ 0 & a3 & a2 & a1 & a2 & a3 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & \dots & 0 & a3 & a2 & a1 \end{bmatrix}$$

Parametry a1, a2 i a3 definiowano na kolejnych etapach projektu.

b to wektor pobudzenia długości 1207, którego n-ty element ma wartość sin(10*n)[1].

Etap 1 - Porównanie metod Jacobiego i Gaussa–Seidla

Przyjęto następujące wartości parametrów macierzy A:

a1 = 10, a2 = -1, a3 = -1.

Dokonano porównania liczby iteracji i czasu działania potrzebnych do wyznaczenia rozwiązania równania, oraz funkcji zmiany wartości normy residuum dla dwóch metod iteracyjnych. Warunkiem zakończenia obliczeń jest osiągnięcie normy residuum mniejszej niż 10^{-9}. Otrzymane dane przedstawiono w formie wykresu.

Etap 2 - Porównanie metod iteracyjnych z faktoryzacją LU

Przyjęto następujące wartości parametrów macierzy A:

a1 = 3, a2 = -1, a3 = -1.

Dokonano analizy zmiany normy residuum dla metod Jacobiego i Gaussa–Seidla.

Następnie dokonano obliczeń metodą rozkładu LU i porównano algorytmy.

Etap 3 - Wyznaczenie zależności czasu obliczeń od wartości N, dla trzech metod

Przyjęto taką samą macierz A jak w etapie 1.

Dla wszystkich metod dokonano pomiaru czasu działania w zależności od wartości N.

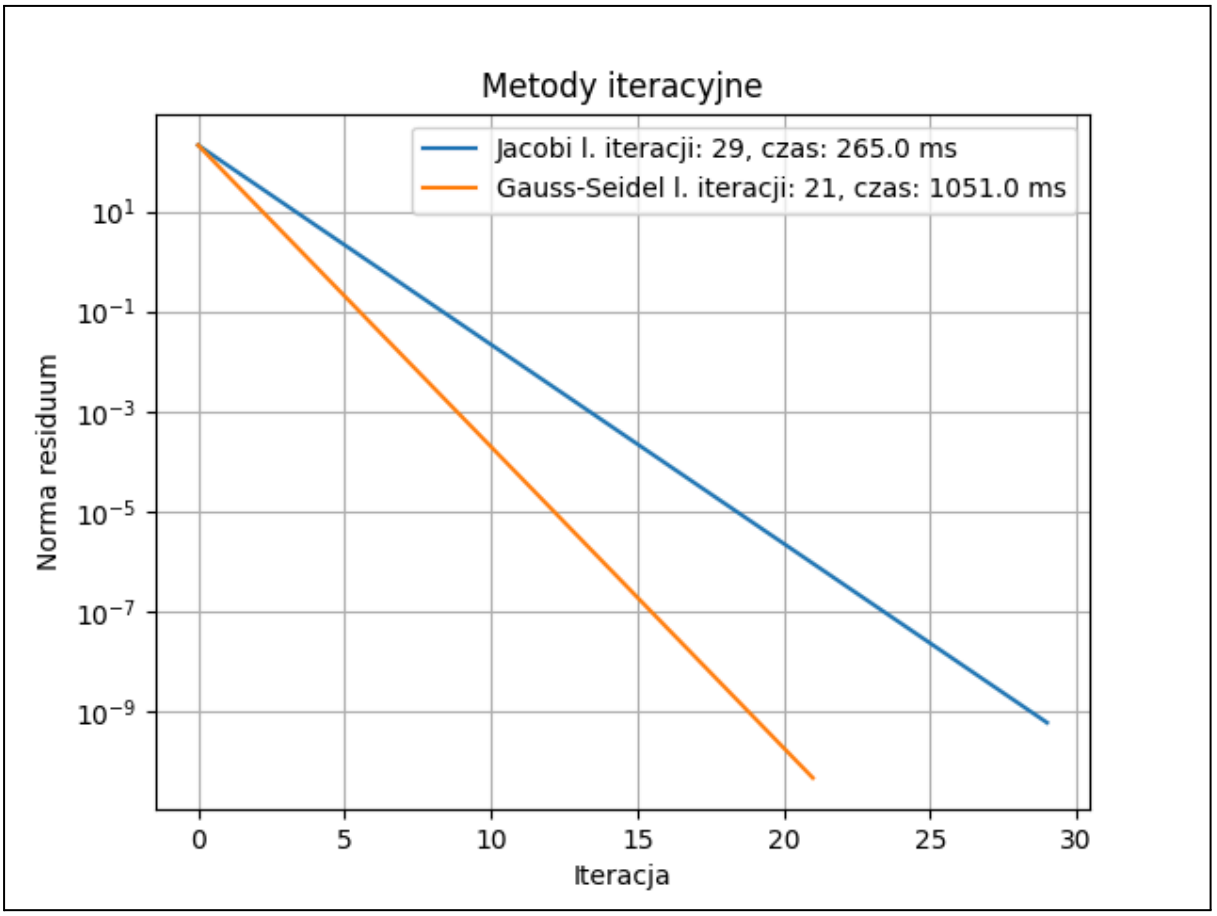
N = {100, 500, 1000, 2000, 3000, 4000}.

Otrzymane dane przedstawiono w formie wykresów.

Wyniki i analiza

1. Etap 1

Po przeprowadzeniu obliczeń metodami iteracyjnymi dla parametrów wejściowych podanych wyżej, stworzono wykres (rys. 1) przedstawiający wartości normy residuum w zależności od iteracji algorytmu.

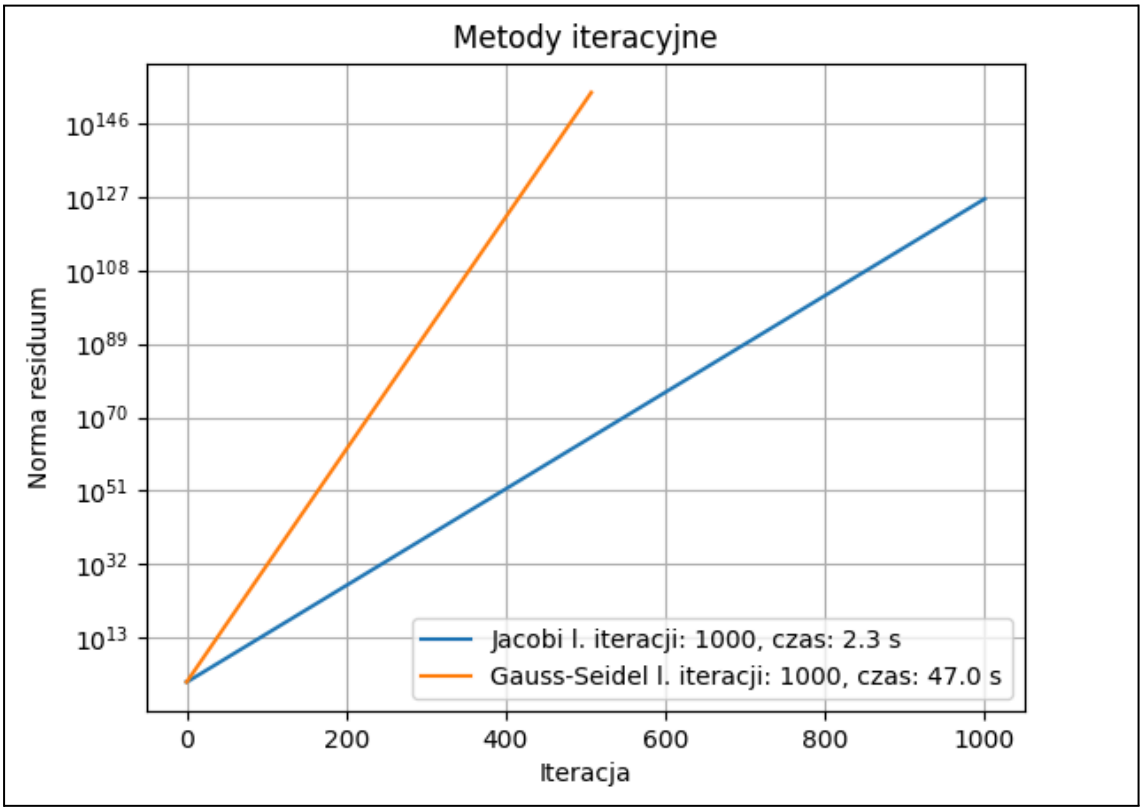


Rys. 1 Wartości normy residuum w kolejnych iteracjach dla metod Jacobiego oraz Gaussa-Seidla - etap1.

Omawiana zależność jest wykładnicza. Funkcja dla metody Gaussa-Seidla maleje bardziej dynamicznie, dlatego otrzymanie zadowalającego wyniku wymagało mniejszej ilości iteracji niż przy metodzie Jacobiego, ale trwało prawie 4 razy dłużej

2. Etap 2

Poniższy wykres (rys. 2.) zawiera wynik działania algorytmów iteracyjnych dla danych wejściowych: *a1=3, a2=a3=-1.*



Rys. 2 Wartości normy residuum w kolejnych iteracjach dla metod Jacobiego oraz Gaussa-Seidla - etap 2.

Wartości normy residuum nie zbiegają się. W przypadku obu metod poszukiwanie rozwiązania zakończyło się po osiągnięciu maksymalnej możliwej ilości iteracji = 1000.
Po raz kolejny można zaobserwować zależność wykładniczą i bardziej dynamiczny wzrost wartości funkcji dla metody Gaussa-Seidla. Różnica w czasie wykonywania programu metodą Jacobiego w porównaniu z danymi z etapu 1 jest niewielka, natomiast dla drugiej metody zaobserwowano drastyczny wzrost długości czasu działania.

W tabeli 1 przedstawiono zestawienie wyników działań poszczególnych algorytmów zarówno iteracyjnych jak i bezpośredniego.

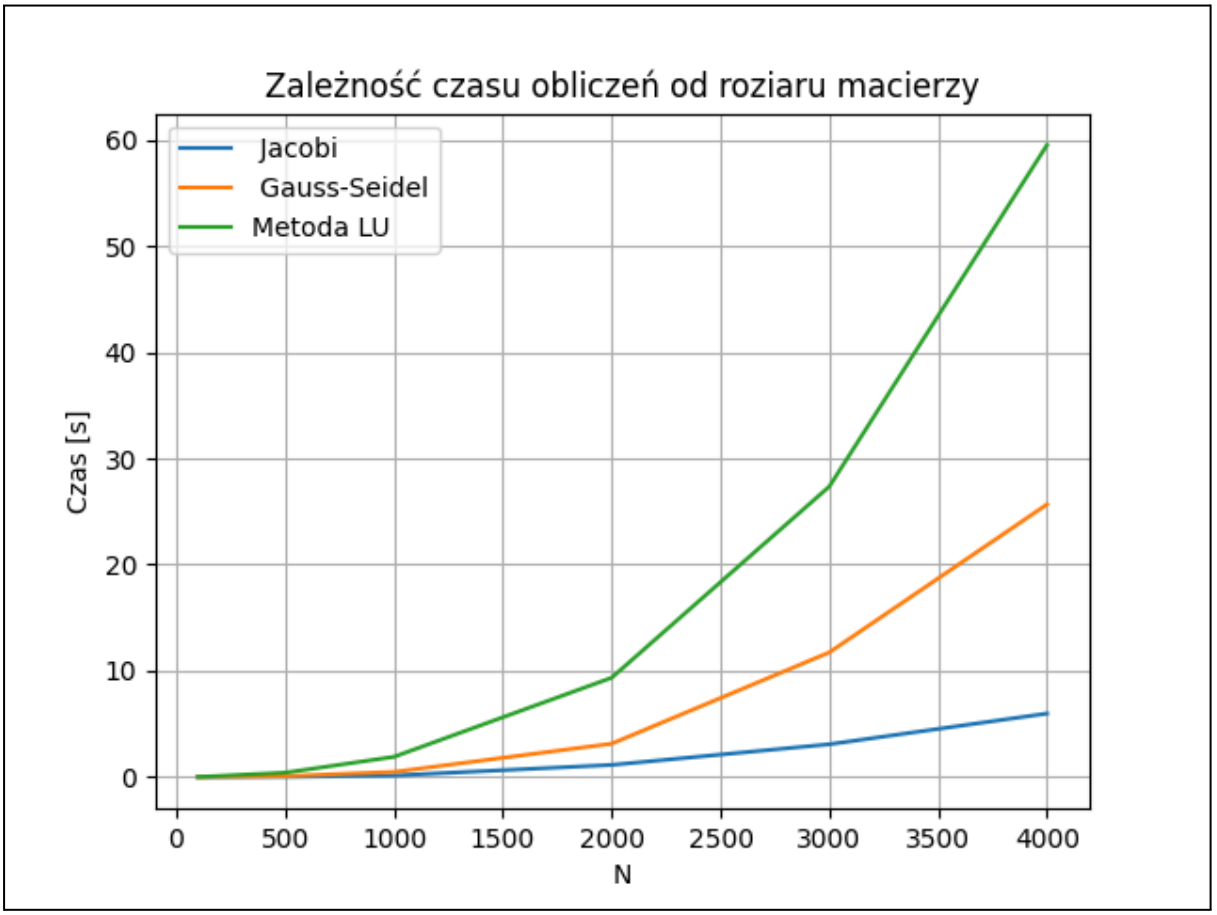
Metoda	Czas działania [s]	Norma residuum
Jacobiego	2.3	$>10^{10}$
Gaussa-Seidla	47.0	$>10^{140}$
Faktoryzacja LU	6.55	$1.85 * 10^{-18}$

Tab. 1. Czas działania i norma residuum dla różnych metod - etap 2.

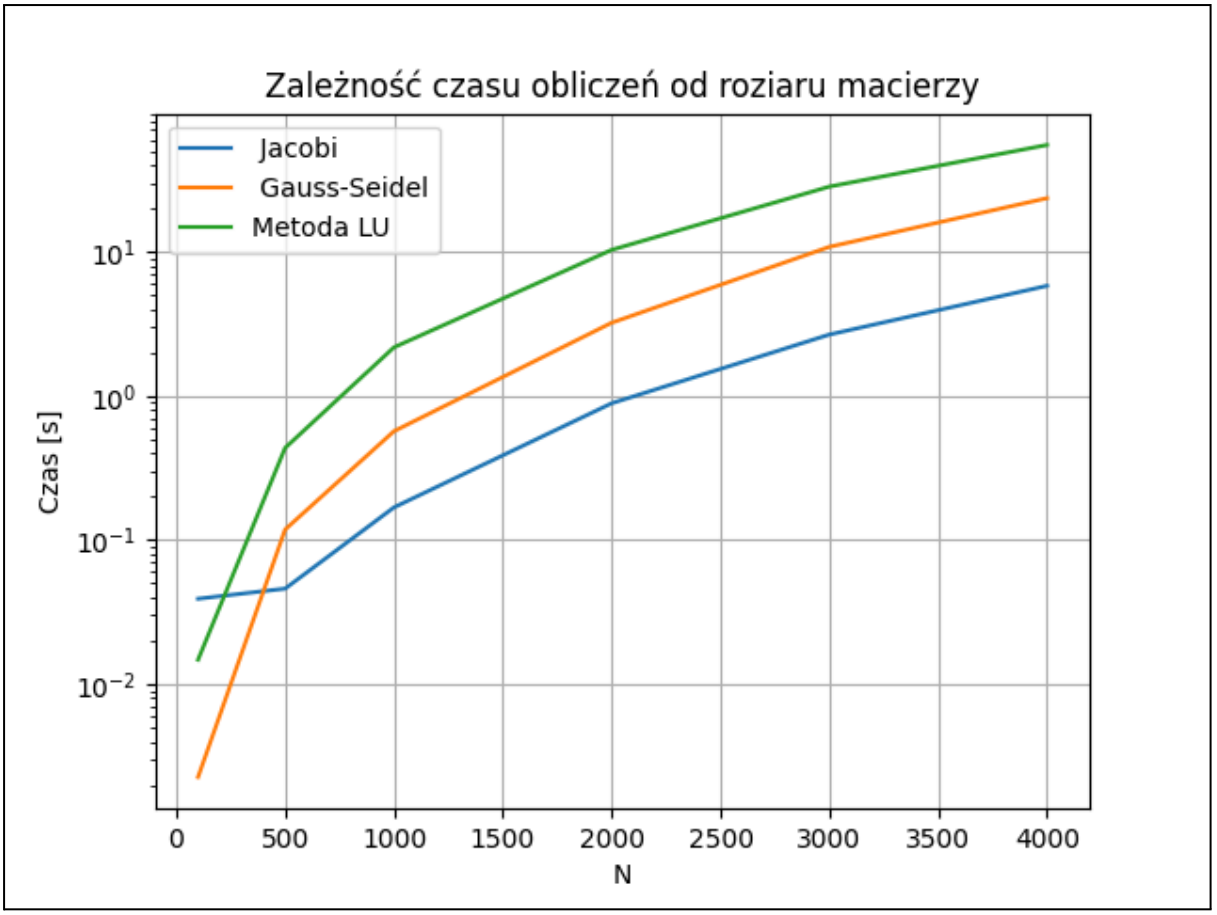
Tylko metoda bezpośredniego rozwiązywania układów równań pozwoliła otrzymać prawidłowy wynik.

3. Etap 3

Na poniższych rysunkach przedstawiono wykresy czasu działania poszczególnych algorytmów dla różnych rozmiarów macierzy. Pierwszy z nich zawiera liniową skalę y (rys.3) natomiast drugi (rys. 4) - skalę logarytmiczną.



Rys. 3. Wykres zależności czasu działania algorytmu od rozmiaru macierzy A (skala liniowa).



Rys. 4. Wykres zależności czasu działania algorytmu od rozmiaru macierzy A (skala logarytmiczna).

Tempo wzrostu oraz czas wykonywania obliczeń jest największy dla metody faktoryzacji LU. Okazała się ona najmniej odporna na zwiększanie rozmiaru danych wejściowych. Wydłużenie czasu działania jest najmniejsze podczas korzystania z metody Jacobiego.

Wnioski

Na podstawie etapu 1. projektu można stwierdzić, że dla omawianego problemu i implementacji, korzyści z równoległości Jacobiego przeważają nad potencjalnie większą liczbą iteracji, których może potrzebować. Ogólny zwycięzca pod względem całkowitego czasu wykonania zależy od konkretnego problemu (macierz A), rozmiaru systemu (N) i tego, jak skutecznie biblioteki bazowe mogą wykorzystać równoległość dla każdej metody

Wyniki etapu 2. wyraźnie pokazują, że metody iteracyjne w przeciwieństwie do bezpośrednich nie nadają się do wszystkich układów równań. Badana macierz A nie spełniała warunku dominacji diagonalnej. Należy ją jednak stosować jedynie jeśli występuje taka konieczność, ponieważ funkcja ta charakteryzuje się najdłuższym czasem działania.

Bibliografia

[1] Grzegorz Fotyga, *Metody Numeryczne Projekt 2 – Układy równań liniowych,* 24 marca 2025

[2] Grzegorz Fotyga, Piotr Sypek, *Metody numeryczne, laboratorium 3. Metody rozwiązywania równania macierzowego. 22 marca 2025*

[3] https://pl.wikipedia.org/wiki/Metoda_LU (10.05.2025)